

OPEN SOURCE NODE.JS ARCHITECTURE

batch-rate-queue

Zero-Redis resilient background worker library combining high-precision token-bucket rate limiting with high-throughput in-memory batch SQL write-buffering.

Package: **npm install batch-rate-queue** Runtime: **Node.js >= 16** License: **MIT** Dependencies: **0 Runtime Dependencies**

1 The Problem Landscape: Why Async Background Jobs Fail

Modern backend applications interact with third-party web APIs (Stripe, OpenAI, OpenStreetMap, SendGrid) and write results back to relational databases. In standard Node.js implementations, this simple workflow introduces four crippling production failure modes:

1. The "429 Too Many Requests" Cascade

APIs enforce strict rate limits (e.g., Nominatim allows 1 req/sec; OpenAI enforces TPM/RPM). Naive loops like `Promise.all()` burst hundreds of requests simultaneously, resulting in immediate HTTP 429 bans, dropped tasks, and IP blacklisting.

2. 10,000 Single SQL Writes (DB Exhaustion)

Updating the database row-by-row inside each worker execution floods the connection pool with thousands of sequential roundtrips. Latency explodes, locks collide, CPU spikes to 100%, and incoming API requests face pool timeouts.

3. Heavy Redis & BullMQ Infrastructure Tax

To prevent the above, teams deploy BullMQ or Celery. But this forces companies to provision, monitor, secure, and pay for an external Redis cluster (\$20–\$100+/mo) for simple background tasks that only run inside a single server process.

4. In-Flight Data Loss During Deployments

Cloud platforms (Docker, Railway, Render, Fly.io, Kubernetes) trigger rolling restarts frequently. When a SIGTERM arrives, naive in-memory buffers terminate immediately, permanently losing dozens or hundreds of processed in-flight updates.

2 Why We Made It: The "Zero-Redis" Philosophy

The Missing Middle Ground in Backend Engineering

Developers were stuck between two extremes: **writing fragile ad-hoc loops** that break in production, or **introducing heavy message-broker infrastructure** (Redis, RabbitMQ, BullMQ) that creates massive operational overhead. We created batch-rate-queue to provide enterprise-grade rate-limiting, batching, and graceful resilience **entirely in-process** with zero external dependencies and zero setup friction.

0

EXTERNAL DAEMONS (NO REDIS)

98.5%

REDUCTION IN DB ROUNDTRIPS

< 1 ms

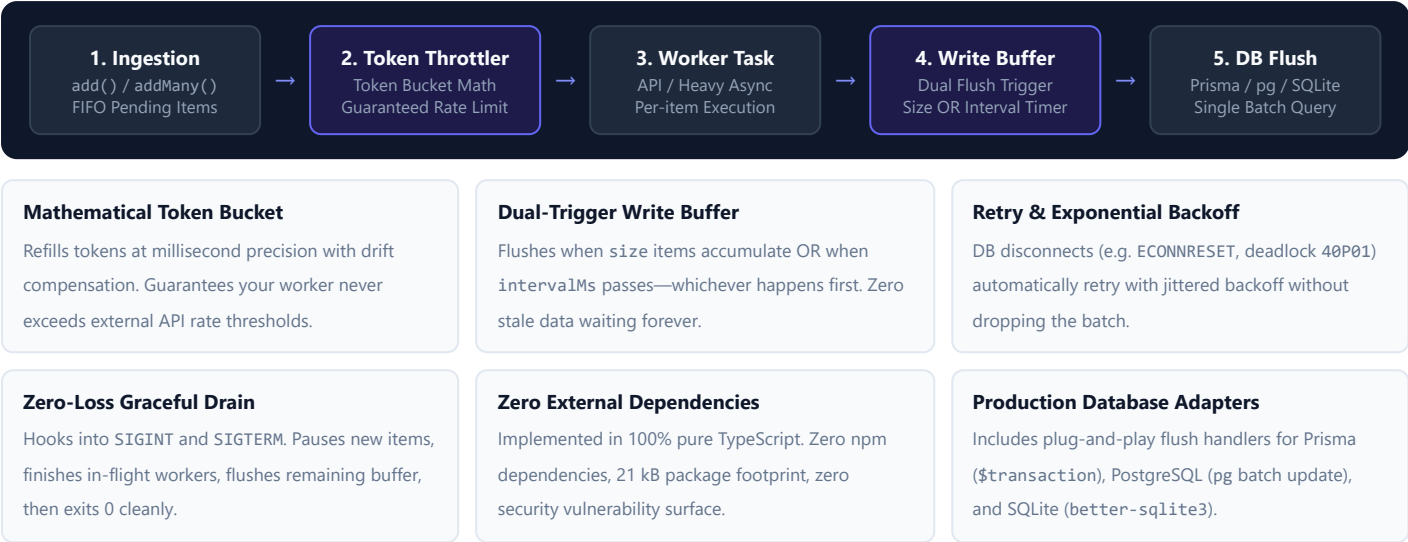
INTERNAL MEMORY QUEUE LATENCY

0 B

IN-FLIGHT DATA LOST ON EXIT

3 Architecture & How It Works

batch-rate-queue seamlessly decouples task ingestion, rate-limited execution, and database write batching into a clean, event-driven pipeline:



5 Step-by-Step Developer Quickstart

Install the package and instantiate a production queue in less than two minutes:

```
terminal
```

```
bash
```

```
npm install batch-rate-queue
```

Real-world example: geocoding 5,000 addresses via Nominatim (1 req/sec limit) while batch-updating PostgreSQL via Prisma in chunks of 50:

```
src/worker.ts
```

```
TypeScript
```

```
import { createBatchRateQueue } from 'batch-rate-queue';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

// 1. Define queue with rate-limit and batching configuration
const queue = createBatchRateQueue({
  name: 'nominatim-enricher',
  ratelimit: { requests: 1, perMs: 1000 }, // Never exceed 1 req/sec
  batchFlush: { size: 50, intervalMs: 5000, maxRetries: 3 },

  // Worker: Executed per item subject to rate limit
  worker: async (item: { id: string; address: string }) => {
    const res = await fetch(`https://nominatim.openstreetmap.org/search?q=${encodeURIComponent(item.address)}&format=json`);
    const [geo] = await res.json();
    return { id: item.id, lat: parseFloat(geo.lat), lon: parseFloat(geo.lon) };
  },

  // Batch Flush: Executed with up to 50 results at once in a single transaction
  onBatchFlush: async (batch) => {
    await prisma.$transaction(
      batch.map(loc => prisma.place.update({
        where: { id: loc.id },
        data: { latitude: loc.lat, longitude: loc.lon }
      }))
    );
  }
});

// 2. Enqueue tasks safely and wait for completion
await queue.addMany(unprocessedLocations);
await queue.waitUntilDrained();
```

6 Lifecycle & Queue Control API

Method / Event	Type	Description
add(item)	Promise<void>	Adds a single task to the FIFO queue.
addMany(items)	Promise<void>	Bulk-adds an array of tasks to the queue.
pause() / resume()	void	Pauses dequeuing or resumes active processing immediately.
waitUntilDrained()	Promise<void>	Resolves only when all tasks are processed AND write buffer is flushed.
getStats()	QueueStats	Returns live counters: processed, failed, pending, buffered.
event: 'batchFlushed'	(batch) => void	Emitted every time a database write batch commits successfully.

7 Pre-Built Database Adapters

batch-rate-queue exports modular, drop-in flush handlers for popular database drivers:

Prisma Adapter

subpath export

```
import { createPrismaFlushHandler }
  from 'batch-rate-queue/adapters/prisma';

const queue = createBatchRateQueue({
  // ...
  onBatchFlush: createPrismaFlushHandler({
    prisma,
    buildOperation: (item) =>
      prisma.user.update({
        where: { id: item.id },
        data: item.changes
      })
  })
});
```

PostgreSQL (pg) Adapter

subpath export

```
import { createPgFlushHandler }
  from 'batch-rate-queue/adapters/pg';

const queue = createBatchRateQueue({
  // ...
  onBatchFlush: createPgFlushHandler({
    pool,
    query: 'UPDATE users SET status = $1 WHERE id = $2',
    mapItemToParams: (u) => [u.status, u.id]
  })
});
```

8 Production Checklist & Best Practices

1. Size vs. Interval Balance

Set size: 50–100 with intervalMs: 2000–5000.
This guarantees timely writes even when task ingestion slows down.

2. Rate Limit Match

Always match provider quotas. If an API allows 60 requests/min, configure { requests: 60, perMs: 60000 }.

3. Graceful Shutdown

Keep gracefulShutdown: true (default) enabled in containers (Kubernetes/Docker) to prevent dropped writes on restarts.

9 Project & Ecosystem Information

Official Open-Source Resources

batch-rate-queue is actively maintained and free under the MIT license.

- **npm Registry:** <https://www.npmjs.com/package/batch-rate-queue>
- **GitHub Repository:** <https://github.com/piyushkumar-prog/batch-rate-queue>
- **Issue Tracker:** <https://github.com/piyushkumar-prog/batch-rate-queue/issues>
- **Author:** Piyush Kumar (@piyushkumar-prog)